

☒ Gr. 1, Dr. D. AuerName Klemens DannerAufwand in h 15☐ Gr. 2, Dr. G. Kronberger☐ Gr. 3, Dr. S. Wagner

Punkte _____

Kurzzeichen Tutor*in / Übungsleiter*in ____ / ____

1. Laufzeitkomplexität**(8 Punkte)**

Zur Berechnung der Summe der Quadrate aller ganzen Zahlen von a bis b (mit $a \leq b$) werden folgende zwei Pascal-Funktionen vorgeschlagen:

```
FUNCTION SumSquares1(a, b: LONGINT): LONGINT;  
  VAR  
    sum, i: LONGINT;  
BEGIN  
  sum := 0;  
  FOR i := a TO b DO BEGIN  
    sum := sum + i * i;  
  END; (* FOR *)  
  SumSquares1 := sum  
END; (* SumSquares1 *)
```

```
FUNCTION SumSquares2(a, b: LONGINT): LONGINT;  
BEGIN  
  IF a < b THEN  
    SumSquares2 := a * a + SumSquares2(a + 1, b)  
  ELSE  
    SumSquares2 := a * a  
  END; (* SumSquares2 *)
```

- a) Führen Sie eine Feinanalyse der zwei Funktionen (in Abhängigkeit von *einer* Problemgröße n) durch. Verwenden Sie dazu folgende relativen Ausführungszeiten:

Operation	Laufzeit
Wertzuweisung	1.0
Vergleich	1.0
Einfache Indizierung	0,5
Addition, Subtraktion	0.5
Multiplikation	3
Division, Restbildung	7.0
Prozeduraufruf	$16 + 2 * \text{Anz. d. Parameter}$

- b) Bestimmen Sie die asymptotische Laufzeitkomplexität der zwei Algorithmen und begründen Sie Ihr Ergebnis.

2. Zweidimensionale Felder sortieren

(8 Punkte)

Gegeben ist eine Matrix mit rows Zeilen und columns Spalten auf Basis folgender Deklarationen.

```
CONST
  rows = ...;
  columns = ...;
TYPE
  Row = ARRAY [1..columns] OF INTEGER;
  Matrix = ARRAY [1..rows] OF Row;
```

Gesucht ist eine Prozedur

```
PROCEDURE SortLinesByColumns(VAR m: Matrix);
```

die die Zeilen der Matrix m anhand der Werte in jeder Zeile (von links nach rechts) mit einem einfachen stabilen Sortierverfahren (z.B. Einfügesortieren) aufsteigend sortiert.

Beispiele:

Beispiel	Unsortierte 5x5-Matrix	Sortierte 5x5-Matrix																																																		
1	<table><tr><td>2</td><td>1</td><td>2</td><td>1</td><td>1</td></tr><tr><td>1</td><td>2</td><td>3</td><td>7</td><td>2</td></tr><tr><td>2</td><td>1</td><td>3</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>3</td><td>4</td><td>5</td></tr><tr><td>2</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	2	1	2	1	1	1	2	3	7	2	2	1	3	1	1	1	1	3	4	5	2	1	1	1	1	<table><tr><td>1</td><td>1</td><td>3</td><td>4</td><td>5</td></tr><tr><td>1</td><td>2</td><td>3</td><td>7</td><td>2</td></tr><tr><td>2</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>2</td><td>1</td><td>2</td><td>1</td><td>1</td></tr><tr><td>2</td><td>1</td><td>3</td><td>1</td><td>1</td></tr></table>	1	1	3	4	5	1	2	3	7	2	2	1	1	1	1	2	1	2	1	1	2	1	3	1	1
2	1	2	1	1																																																
1	2	3	7	2																																																
2	1	3	1	1																																																
1	1	3	4	5																																																
2	1	1	1	1																																																
1	1	3	4	5																																																
1	2	3	7	2																																																
2	1	1	1	1																																																
2	1	2	1	1																																																
2	1	3	1	1																																																
2	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	0	0	1	1	1	1	1	1	1	1	0	0	0	0	1	0	0	0	1	1	0	1	1	1	1	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	0	0	0	0	1	0	0	0	1	1	0	0	1	1	1	0	1	1	1	1	1	1	1	1	1
0	0	1	1	1																																																
1	1	1	1	1																																																
0	0	0	0	1																																																
0	0	0	1	1																																																
0	1	1	1	1																																																
0	0	0	0	1																																																
0	0	0	1	1																																																
0	0	1	1	1																																																
0	1	1	1	1																																																
1	1	1	1	1																																																

3. Auswahlsortieren auf einfach-verkettete Listen

(8 Punkte)

Gegeben ist eine einfach-verkettete Liste list auf Basis folgender Deklarationen.

```
TYPE
  ListNodePtr = ^ListNode;
  ListNode = RECORD
    next: ListNodePtr;
    data: INTEGER;
  END; (* ListNode *)
  ListPtr = ListNodePtr;
```

Gesucht ist eine Prozedur

```
PROCEDURE SelectionSort(VAR list: ListPtr);
```

die die Liste list mittels Auswahlsortieren aufsteigend sortiert.

Hinweise:

1. Geben Sie für alle Ihre Lösungen immer eine „Lösungsidee“ an.
2. Dokumentieren und kommentieren Sie Ihre Pascal-Programme.
3. Geben Sie immer auch Testfälle ab, an denen man erkennen kann, dass Ihr Pascal-Programm funktioniert, und dass es auch in Fehlersituationen entsprechend reagiert.

1 Laufzeitkomplexität

a)

Sei n in diesem Fall die Problemgröße:
 $n := (b-a) + 1$

Algorithmus	Anz.	Gewicht	n	C
FUNCTION SumSquares1(a, b: LONGINT): LONGINT;				
VAR				
sum, i: LONGINT;				
BEGIN				
sum := 0;	1	1,0		1
i := a;	1	1,0		1
while i <= b do begin	$n+1$	1,0	1	1
sum := sum + i * i;	n	$1+0,5+3=4,5$	4,5	
i := i + 1;	n	$1+0,5=1,5$	1,5	
end;				
SumSquares1 := sum	1	1,0		1
END;				

$$T(n) = 7n + 4$$

Algorithmus 2:

Problemgröße $n := (b-a) + 1$

	Gewicht	Anz.	n	C
FUNCTION SumSquares2(a, b: LONGINT): LONGINT;				
BEGIN				
IF a < b THEN	1	$1,0 \cdot (n-1) = n-1$	1	-1
SumSquares2 := a * a + SumSquares2(a + 1, b)	1	$(1+3+0,5+2+0,5) \cdot (n-1)$ $= 25(n-1)$ $= 25n - 25$	25	-25
ELSE				
SumSquares2 := a * a	1+Vergl.	$1+3+1=5$	0	5
END; (* SumSquares2 *)				

nicht-rekursiver
Vergleich; wenn $n(a < b)$

$$T_2(n) = 26n - 21$$

b) Asymptotische Laufzeitkomplexität

Die asymptotische Laufzeitkomplexität lässt sich einfach aus den Laufzeitfunktionen ablesen. Da die höchste Potenz jeweils 1 beträgt, ist die asymptotische Laufzeitkomplexität jeweils $O(n)$.

Trotzdem ist der iterative Algorithmus natürlich schneller.

2 Zweidimensionale Felder sortieren

2.1 Lösungsidee

In dieser Aufgabe sollen die Zeilen sortiert werden. Man braucht zunächst eine Möglichkeit, Zeilen zu vergleichen. Dazu wird eine Funktion `RowGreaterThan` definiert, welche zwei Zeilen vergleicht, in dem sie zunächst alle gleichen Werte überspringt und dann den nächsten Wert vergleicht. Diese Funktion lässt sich einfach implementieren, weil nur gleich lange Zeilen verglichen werden sollen.

Die Prozedur `SortLinesByColumns` implementiert ein normales Einfügesortieren. Dabei wird das nächste Element immer in einen sortierten Bereich eingefügt (hier von `first` bis `i`).

Zunächst wird das erste Element, welches nicht im sortierten Bereich liegt gespeichert, werden die Elemente im sortierten Bereich in einer `while`-Schleife so lange nach rechts verschoben, bis die Einfügeposition gefunden wurde.

2.2 Quellcode

```
1  PROGRAM SortMatrix;
2
3  CONST
4      rows = 5;
5      columns = 5;
6
7  TYPE
8      Row = ARRAY[1..columns] OF INTEGER;
9      Matrix = ARRAY[1..rows] OF Row;
10
11  PROCEDURE ReadMatrix(VAR m: Matrix);
12      VAR
13          i, j: INTEGER;
14  BEGIN
15      FOR i := 1 TO rows DO BEGIN
16          Write('Enter a row with ', columns, ' columns > ');
17          FOR j := 1 TO columns DO BEGIN
18              Read(m[i][j]);
19          END;
20      END;
21  END;
22
23  PROCEDURE WriteMatrix(m: Matrix);
24      VAR
25          i, j: INTEGER;
26  BEGIN
27      FOR i := 1 TO rows DO BEGIN
28          Write('[ ');
29          FOR j := 1 TO columns DO BEGIN
30              Write(m[i][j]:2);
31          END;
32          Write(' ]');
33          WriteLn;
34      END;
35  END;
36
37
38  FUNCTION RowGreaterThan(a, b: Row): BOOLEAN; (*returns true if a > b*)
39      VAR
40          len, i: INTEGER;
41  BEGIN
42      i := 1;
43      len := length(a);
```

```

44
45     WHILE (i <= len) AND (a[i] = b[i]) DO BEGIN
46         inc(i);
47     END;
48
49     RowGreaterThan := (i <= len) AND (a[i] > b[i]);
50 END;
51
52 PROCEDURE SortLinesByColumns(VAR m: Matrix);
53     VAR
54         first, last: INTEGER;
55         i, j: INTEGER;
56         tmp: Row;
57 BEGIN
58     first := 1;
59     last := rows;
60
61     FOR i := first TO last - 1 DO BEGIN
62         (* sorted area: first .. i*)
63         tmp := m[i + 1];
64         j := i;
65         WHILE (j >= first) AND RowGreaterThan(m[j], tmp) DO BEGIN
66             m[j + 1] := m[j];
67             j := j - 1;
68         END;
69         m[j + 1] := tmp;
70     END;
71 END;
72
73 VAR
74     m: Matrix;
75
76 BEGIN
77     ReadMatrix(m);
78     WriteLn;
79     WriteMatrix(m);
80     WriteLn;
81
82     SortLinesByColumns(m);
83     WriteMatrix(m);
84     WriteLn;
85 END.

```

2.3 Tests

Anmerkung	Input / Output
Beispiel 0	<pre>Enter a row with 5 columns > 2 1 2 1 1 Enter a row with 5 columns > 1 2 3 7 2 Enter a row with 5 columns > 2 1 3 1 1 Enter a row with 5 columns > 1 1 3 4 5 Enter a row with 5 columns > 2 1 1 1 1 [2 1 2 1 1] [1 2 3 7 2] [2 1 3 1 1] [1 1 3 4 5] [2 1 1 1 1] [1 1 3 4 5] [1 2 3 7 2] [2 1 1 1 1] [2 1 2 1 1] [2 1 3 1 1]</pre>
Beispiel 1	<pre>Enter a row with 5 columns > 0 0 1 1 1 Enter a row with 5 columns > 1 1 1 1 1 Enter a row with 5 columns > 0 0 0 0 1 Enter a row with 5 columns > 0 0 0 1 1 Enter a row with 5 columns > 0 1 1 1 1 [0 0 1 1 1] [1 1 1 1 1] [0 0 0 0 1] [0 0 0 1 1] [0 1 1 1 1] [0 0 0 0 1] [0 0 0 1 1] [0 0 1 1 1] [0 1 1 1 1] [1 1 1 1 1]</pre>

Anmerkung	Input / Output
1. 1 1 1 1 1 und 1 1 1 1 2 beweisen, dass die Vergleichsfunktion bis zur letzten Ziffer vergleicht 2. 1 9 9 9 9 und 2 0 0 0 0 zeigen, dass der erste auftretende Unterschied ausschlaggebend ist	<pre> > Enter a row with 5 columns > 2 0 0 0 0 Enter a row with 5 columns > 1 1 1 1 2 Enter a row with 5 columns > 1 9 9 9 9 Enter a row with 5 columns > 1 1 1 1 1 Enter a row with 5 columns > 0 0 0 0 0 [2 0 0 0 0] [1 1 1 1 2] [1 9 9 9 9] [1 1 1 1 1] [0 0 0 0 0] [0 0 0 0 0] [1 1 1 1 1] [1 1 1 1 2] [1 9 9 9 9] [2 0 0 0 0] </pre>
bereits sortiert	<pre> > Enter a row with 5 columns > 1 2 3 4 5 Enter a row with 5 columns > 2 3 4 5 6 Enter a row with 5 columns > 3 4 5 6 7 Enter a row with 5 columns > 4 5 6 7 8 Enter a row with 5 columns > 5 6 7 8 9 [1 2 3 4 5] [2 3 4 5 6] [3 4 5 6 7] [4 5 6 7 8] [5 6 7 8 9] [1 2 3 4 5] [2 3 4 5 6] [3 4 5 6 7] [4 5 6 7 8] [5 6 7 8 9] </pre>

Anmerkung	Input / Output
umgekehrt sortiert	<pre>Enter a row with 5 columns > 5 6 7 8 9 Enter a row with 5 columns > 4 5 6 7 8 Enter a row with 5 columns > 3 4 5 6 7 Enter a row with 5 columns > 2 3 4 5 6 Enter a row with 5 columns > 1 2 3 4 5 [5 6 7 8 9] [4 5 6 7 8] [3 4 5 6 7] [2 3 4 5 6] [1 2 3 4 5] [1 2 3 4 5] [2 3 4 5 6] [3 4 5 6 7] [4 5 6 7 8] [5 6 7 8 9]</pre>

3 Auswahlsortieren auf einfach-verkettete Listen

3.1 Lösungsidee

Bei der Implimentierung eines SelectionSorts für eine einfach verkettete Liste ergeben sich einige Besonderheiten im Unterschied zur Anwendung des Algorithmus auf Arrays.

Der größte Unterschied ergibt sich darin, dass man Werte nicht so einfach vertauschen kann, weil dabei der next-Wert des Vorgängers angepasst werden muss. Da man in einer einfach verketteten Liste nicht auf den Vorgänger zugreifen kann, wird hier direkt mit den Vorgängern der eigentlichen Knoten gearbeitet.

In der Prozedur SelectionSort wird die Liste an einen Dummy-Knoten angehängt, damit gegebenenfalls auch das erste Element getauscht werden kann.

Dann wird die Liste in einer Schleife iteriert, falls ein Knoten mit einem kleineren Datenwert gefunden wird, wird er mit dem aktuellen Knoten vertauscht. (mithilfe der Prozedur SwapChildNodes)

Dabei muss der Sonderfall berücksichtigt werden, wenn die zu vertauschenden Knoten nebeneinander liegen, da dann der Parentknoten des zweiten Wertes mit dem Knoten des ersten Wertes ident ist.

3.2 Quellcode

```
1  PROGRAM SelSort;
2
3  TYPE
4      ListNodePtr = ^ListNode;
5      ListNode = RECORD
6          next: ListNodePtr;
7          data: INTEGER;
8      END;
9      ListPtr = ListNodePtr;
10
11  PROCEDURE AddNode(VAR list: ListPtr; val: INTEGER);
12      VAR
13          newNode: ListNodePtr;
14          last: ListNodePtr;
15  BEGIN
16      New(newNode);
17      newNode^.data := val;
18      newNode^.next := NIL;
19
20      IF (list = NIL) THEN
21          list := newNode
22      ELSE BEGIN
23          last := list;
24
25          WHILE (last^.next <> NIL) DO BEGIN
26              last := last^.next;
27          END;
28
29          last^.next := newNode;
30      END;
31  END;
32
33  PROCEDURE ReadList(VAR l: ListPtr);
34      VAR
35          val: INTEGER;
36  BEGIN
37      WriteLn('Enter some int values for your list (end with 0): ');
38      Write('> '); ReadLn(val);
39      WHILE (val <> 0) DO BEGIN
40          AddNode(l, val);
41          Write('> '); ReadLn(val);
42      END;
43
```

```

44     END;
45
46     PROCEDURE DisposeList(VAR l: ListPtr);
47         VAR
48             curr, pt: ListNodePtr;
49     BEGIN
50         curr := l;
51         WHILE curr <> NIL DO BEGIN
52             pt := curr;
53             curr := curr^.next;
54             Dispose(pt);
55             pt := NIL;
56         END;
57     END;
58
59     PROCEDURE WriteList(l: ListPtr);
60         VAR
61             curr: ListNodePtr;
62     BEGIN
63         curr := l;
64         WHILE curr <> NIL DO BEGIN
65             Write(curr^.data:3);
66             curr := curr^.next;
67         END;
68     END;
69
70     FUNCTION SmallestNodePt(l: ListPtr): ListNodePtr; (*returns parent of
the smallest node*)
71         VAR
72             minPtr, curr, minParent, pt: ListNodePtr;
73     BEGIN
74         minPtr := l;
75         minParent := NIL;
76         curr := l;
77         pt := NIL;
78         WHILE curr <> NIL DO BEGIN
79             IF curr^.data < minPtr^.data THEN BEGIN
80                 minParent := pt;
81                 minPtr := curr;
82             END;
83             pt := curr;
84             curr := curr^.next;
85         END;
86         SmallestNodePt := minParent;
87     END;
88

```

```

89     PROCEDURE SwapChildNodes(ptA, ptB: ListNodePtr); (*swaps Nodes using
their parents*)
90     VAR
91         a, b, childOfA: ListNodePtr;
92     BEGIN
93         IF (ptA = NIL) OR (ptB = NIL) OR (ptA^.next = NIL) OR (ptB^.next =
NIL) THEN BEGIN
94             WriteLn('Error: you cannot swap these nodes');
95             Exit;
96         END;
97         a := ptA^.next;
98         b := ptB^.next;
99
100        (*Swapping Case 1: nodes to be swapped are neighbours*)
101        IF (ptB = a) THEN BEGIN
102            a^.next := b^.next;
103            b^.next := a;
104            ptA^.next := b;
105        END ELSE IF (ptA = b) THEN BEGIN
106            b^.next := a^.next;
107            a^.next := b;
108            ptB^.next := a;
109        END ELSE BEGIN (*general case*)
110            childOfA := a^.next;
111
112            ptB^.next := a;
113            a^.next := b^.next;
114
115            ptA^.next := b;
116            b^.next := childOfA;
117        END;
118    END;
119
120    PROCEDURE SelectionSort(VAR list: ListPtr);
121    VAR
122        pt: ListNodePtr;
123        ptB, dummy: ListNodePtr;
124    BEGIN
125        IF (list = NIL) OR (list^.next = NIL) THEN BEGIN
126            WriteLn('List is empty or has only one node');
127            Exit;
128        END;
129
130
131        (* creating a parent dummy node*)
132        New(dummy);

```

```

133     dummy^.next := list;
134     dummy^.data := -1;
135
136     pt := dummy;
137
138     WHILE (pt^.next <> NIL) DO BEGIN
139         ptB := SmallestNodePt(pt^.next);
140         IF ptB <> NIL THEN BEGIN (* smaller node found - swap*)
141             SwapChildNodes(pt, ptB);
142         END;
143         pt := pt^.next;
144     END;
145
146     list := dummy^.next;
147     Dispose(dummy);
148     dummy := NIL;
149
150 END;
151
152 VAR
153     l: ListPtr;
154
155 BEGIN
156     l := NIL;
157     ReadList(l);
158     IF l <> NIL THEN BEGIN
159         Write('unsorted >');
160         WriteList(l);
161         WriteLn;
162     END;
163
164     SelectionSort(l);
165     IF l <> NIL THEN BEGIN
166         Write('sorted >');
167         WriteList(l);
168         WriteLn;
169     END;
170     DisposeList(l);
171 END.

```

3.3 Tests

Anmerkung	Input / Output
5 und 1 sind Nachbarn - edge cases	<pre>○ Enter some int values for your list (end with 0): > 5 > 1 > 8 > 3 > 0 unsorted > 5 1 8 3 sorted > 1 3 5 8</pre>
1 muss von ganz hinten nach ganz vorne, Minimum am Ende	<pre>○ Enter some int values for your list (end with 0): > 5 > 8 > 9 > 7 > 1 > 0 unsorted > 5 8 9 7 1 sorted > 1 5 7 8 9</pre>
umgekehrt sortiert	<pre>○ Enter some int values for your list (end with 0): > 5 > 4 > 3 > 2 > 1 > 0 unsorted > 5 4 3 2 1 sorted > 1 2 3 4 5</pre>
bereits sortiert	<pre>○ Enter some int values for your list (end with 0): > 1 > 2 > 3 > 4 > 5 > 0 unsorted > 1 2 3 4 5 sorted > 1 2 3 4 5</pre>
Liste leer	<pre>○ Enter some int values for your list (end with 0): > 0 List is empty or has only one node</pre>
Nur ein Knoten	<pre>○ Enter some int values for your list (end with 0): > 1 > 0 unsorted > 1 List is empty or has only one node sorted > 1</pre>